



# Avanew Command Center

## Version 1 — Technical Specification & Reproduction Guide

---

|                   |                                       |
|-------------------|---------------------------------------|
| CLASSIFICATION    | Internal Technical Reference          |
| VERSION           | 1.0.0                                 |
| DATE              | April 28, 2026                        |
| PRODUCT           | Avanew Command Center (CRM)           |
| REPOSITORY TARGET | GitHub → AWS EC2                      |
| PREPARED FOR      | Engineering, AI-Assisted Reproduction |
| STATUS            | RELEASED — Version 1.0.0              |

---

# Avanew Command Center

---

## Version 1 — Technical Specification & Reproduction Guide

---

**Document Classification:** Internal Technical Reference

**Version:** 1.0.0

**Date:** April 28, 2026

**Product:** Avanew Command Center (CRM)

**Repository Target:** GitHub → AWS EC2

**Prepared For:** Engineering, AI-Assisted Reproduction

---

### Table of Contents

---

- 1. [Executive Summary](#)
  - 2. [Product Overview](#)
  - 3. [Technical Stack](#)
  - 4. [System Architecture](#)
  - 5. [Database Design](#)
  - 6. [Role & Permission System](#)
  - 7. [Module Specifications](#)
  - 8. [Settings Module](#)
  - 9. [Edge Functions & Backend Services](#)
  - 10. [Key Components & Hooks](#)
  - 11. [AVE — AI Voice Experience](#)
  - 12. [Environment Configuration](#)
  - 13. [Deployment Guide](#)
  - 14. [Master Reproduction Prompt](#)
-

# 1. Executive Summary

---

Avanew Command Center is a full-stack CRM and business intelligence platform purpose-built for a single organization. It combines traditional CRM workflows (leads, contacts, accounts, deals, activities, tasks) with integrated financial data (Cashflow module via Mercury bank and Plaid), AI-powered insights, an AI Voice Experience (AVE powered by Tavus), and a sophisticated role-based access control system.

The platform is built with a modern React frontend served via Vite, a Supabase backend (PostgreSQL + Auth + Edge Functions + Row Level Security), and deployed to AWS EC2. A preview mode ( `VITE_PREVIEW_MODE=true` ) enables full UI testing without a live Supabase instance.

## Version 1 Scope:

- 7-tier role hierarchy with a 35-key permissions matrix
  - 13 CRM/business pages (Leads, Contacts, Accounts, Deals, Activities, Tasks, Reports, Partners, Vendors, Cashflow, Transactions, Bank Connections, Dashboard)
  - Settings module with 14 child routes
  - 4 Supabase Edge Functions with centralized auth middleware
  - AVE voice assistant (Tavus, audio-only)
  - Mercury and Plaid bank sync
- 

# 2. Product Overview

---

## 2.1 Purpose

Avanew Command Center serves as the internal operating system for a business, combining CRM pipeline management, financial intelligence, team coordination, and AI-assisted insights into a single unified platform. It is intentionally scoped to one organization — there are no multi-tenant concerns.

## 2.2 Core Feature Set

| Feature Area        | Description                                                     |
|---------------------|-----------------------------------------------------------------|
| CRM                 | Leads, Contacts, Accounts (Companies), Deals, Activities, Tasks |
| Cashflow            | Financial dashboard, transaction ledger, bank connections       |
| Bank Sync           | Mercury API + Plaid Link for automatic transaction import       |
| Reports             | Business summaries and analytics                                |
| Partners            | External partner relationship management                        |
| Vendors             | Vendor relationship management                                  |
| Settings            | 14-section settings module with nested routing                  |
| Roles & Permissions | 7 roles × 35 permission keys with live-editable matrix          |
| AVE                 | Tavus-powered AI Voice Experience (audio-only, context-aware)   |
| Notifications       | Role-scoped alert system                                        |
| Security            | MFA capability, login history, session management               |
| Audit Logs          | Immutable activity history (Owner+)                             |

## 2.3 Design Principles

1. **Role-first design** — every feature, route, and API call enforces role-based access at all four layers: UI, Route, API, and Data.
2. **No full-width layouts** — all forms and content use controlled max-width containers.
3. **Preview mode** — the full UI is operable without a live backend via `VITE_PREVIEW_MODE=true`.
4. **Minimal surface area** — no external state management libraries; React hooks + Supabase client only.
5. **Edge-first backend** — all sensitive operations (bank sync, auth, AI sessions) run in Supabase Edge Functions, never in the browser.

### 3. Technical Stack

#### 3.1 Frontend

| Layer               | Technology       | Version / Notes                                                                                                                |
|---------------------|------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Framework           | React            | 19                                                                                                                             |
| Language            | TypeScript       | Strict mode                                                                                                                    |
| Build Tool          | Vite             | v5+                                                                                                                            |
| CSS                 | Tailwind CSS     | v4                                                                                                                             |
| Component Library   | shadcn/ui        | Card, Button, Badge, Dialog, Sheet, Tabs, Select, Input, Label, Avatar, Tooltip, AlertDialog, Separator, Textarea, Table, Form |
| Routing             | React Router DOM | v6, nested routes                                                                                                              |
| Icons               | Lucide React     | —                                                                                                                              |
| Toast Notifications | Sonner           | —                                                                                                                              |
| Drag & Drop         | @dnd-kit         | Pipeline kanban                                                                                                                |

#### 3.2 Backend

| Layer              | Technology          | Notes                           |
|--------------------|---------------------|---------------------------------|
| Database           | Supabase PostgreSQL |                                 |
| Authentication     | Supabase Auth       | Email/password + (optional) MFA |
| Edge Functions     | Deno (Supabase)     | TypeScript runtime              |
| Row Level Security | PostgreSQL RLS      | Enforced on all tables          |
| Storage            | Supabase Storage    | (future)                        |

### 3.3 External Integrations

| Service      | Purpose                            | Auth Method          |
|--------------|------------------------------------|----------------------|
| Mercury Bank | Business bank account sync         | API Key (read-only)  |
| Plaid        | Multi-institution bank connections | OAuth + access_token |
| Tavus        | AVE AI Voice replica               | API Key              |
| Twilio       | SMS notifications (future)         | API Key + Secret     |

### 3.4 Project Structure

```
AvanewCRM/
├─ src/
│   ├─ components/           # Shared UI components
│   │   ├─ ui/               # shadcn/ui primitives
│   │   ├─ AppLayout.tsx     # Root layout (sidebar + topbar + outlet)
│   │   ├─ AppSidebar.tsx    # Main navigation sidebar
│   │   ├─ TopBar.tsx        # Header bar
│   │   ├─ RoleGate.tsx      # Route-level access control
│   │   ├─ ProtectedRoute.tsx # Auth guard
│   │   ├─ PageHeader.tsx    # Reusable page header
│   │   ├─ AVE.tsx           # AI Voice Experience FAB
│   │   ├─ StageManager.tsx  # Pipeline stage editor dialog
│   │   └─ TeamSection.tsx   # Team management component
│   ├─ hooks/
│   │   ├─ useRole.ts        # Current user role
│   │   ├─ usePermissions.ts # Matrix-based permissions
│   │   └─ useAVE.ts         # AVE session state machine
│   ├─ lib/
│   │   ├─ supabase.ts       # Supabase client singleton
│   │   ├─ data.ts           # All data CRUD functions
│   │   ├─ permissions.ts    # Permission matrix logic + DB persistence
│   │   ├─ transaction-classifier.ts # Bank transaction categorization
│   │   └─ utils.ts          # cn() and utilities
│   └─ pages/
│       ├─ auth/              # Login, Signup
│       └─ settings/          # 15-file settings module
│           ├─ SettingsLayout.tsx
│           ├─ SettingsProfile.tsx
│           ├─ SettingsRoles.tsx
│           ├─ SettingsCompany.tsx
│           ├─ SettingsTeam.tsx
│           ├─ SettingsPipeline.tsx
│           ├─ SettingsFinancial.tsx
│           ├─ SettingsPartnersVendors.tsx
│           ├─ SettingsIntegrations.tsx
│           ├─ SettingsNotifications.tsx
│           ├─ SettingsSecurity.tsx
│           ├─ SettingsAuditLogs.tsx
│           ├─ SettingsData.tsx
│           ├─ SettingsBranding.tsx
│           └─ SettingsSystem.tsx
│       ├─ Dashboard.tsx
│       ├─ Leads.tsx / LeadForm.tsx
│       ├─ Contacts.tsx / ContactForm.tsx
│       ├─ Companies.tsx / AccountForm.tsx
│       ├─ Deals.tsx / DealForm.tsx
│       ├─ Activities.tsx
│       ├─ Tasks.tsx / TaskForm.tsx
│       └─ Reports.tsx
```



```

| | | └─ Cashflow.tsx
| | | └─ Transactions.tsx / TransactionForm.tsx
| | | └─ BankConnections.tsx
| | | └─ Partners.tsx / PartnerForm.tsx
| | | └─ Vendors.tsx / VendorForm.tsx
| | | └─ PermissionsMatrix.tsx
| | └─ types/
| |   └─ db.ts          # All TypeScript type definitions
| └─ App.tsx          # Root router
└─ supabase/
  └─ functions/
    └─ _shared/
      └─ auth.ts      # requireAuth() middleware
      └─ cors.ts      # CORS headers
    └─ mercury-sync/  # Mercury bank sync
    └─ plaid-link-token/ # Plaid OAuth step 1
    └─ plaid-sync/    # Plaid exchange + sync
    └─ ave-session/   # Tavus conversation management
  └─ schema.sql       # Complete database schema + RLS
└─ .env               # VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY
└─ .env.local         # VITE_PREVIEW_MODE=true (dev only)
└─ AVANEW_CRM_V1_SPECIFICATION.md

```

## 4. System Architecture

### 4.1 Application Layers



### 4.2 Four-Layer Enforcement Model

Every sensitive feature is protected at all four layers:

| Layer | Mechanism                                                | Example                     |
|-------|----------------------------------------------------------|-----------------------------|
| UI    | <code>can(key)</code> from <code>usePermissions</code>   | Hide cashflow nav link      |
| Route | <code>&lt;RoleGate permission="cashflow.view"&gt;</code> | Redirect unauthorized users |
| API   | <code>requireAuth(req, ["owner", "super_user"])</code>   | Return 403 in edge function |
| Data  | PostgreSQL RLS policies                                  | Filter rows by role/user    |

### 4.3 Preview Mode

When `VITE_PREVIEW_MODE=true`:

- Authentication is bypassed; role defaults to `super_user`
- Supabase calls are intercepted; mock data is returned from `src/lib/data.ts`
- AVE shows a simulated 1.4-second connection delay then fake active state

- Permissions matrix uses `localStorage` instead of Supabase DB
  - All UI branches render as if fully authenticated
- 

## 5. Database Design

---

### 5.1 Core Types

```
-- Role enum (7 tiers)
create type public.team_role as enum (
    'super_user', 'owner', 'admin', 'member', 'viewer', 'bd', 'partner'
);
```

## 5.2 Table Reference

| Table                               | Purpose                            | RLS                                                  |
|-------------------------------------|------------------------------------|------------------------------------------------------|
| <code>profiles</code>               | User profiles linked to auth.users | <code>is_crm_reader()</code>                         |
| <code>leads</code>                  | Sales leads                        | <code>is_crm_reader()</code> + BD assigned           |
| <code>contacts</code>               | CRM contacts                       | <code>is_crm_reader()</code> + BD assigned           |
| <code>companies</code>              | Accounts / companies               | <code>is_crm_reader()</code> + BD assigned           |
| <code>deals</code>                  | Sales deals                        | <code>is_crm_reader()</code> + BD assigned           |
| <code>activities</code>             | Call/email/meeting logs            | <code>is_crm_reader()</code> + BD assigned           |
| <code>tasks</code>                  | Task management                    | <code>is_crm_reader()</code> + BD assigned           |
| <code>pipeline_stages</code>        | Deal stage configuration           | <code>is_crm_reader()</code> or <code>is_bd()</code> |
| <code>partners</code>               | Partner records                    | <code>is_crm_reader()</code> or <code>is_bd()</code> |
| <code>vendors</code>                | Vendor records                     | <code>is_crm_reader()</code> or <code>is_bd()</code> |
| <code>cashflow_transactions</code>  | Manual cashflow entries            | <code>is_owner()</code> only                         |
| <code>bank_connections</code>       | Mercury/Plaid connection records   | <code>is_owner()</code> only                         |
| <code>bank_connection_tokens</code> | Plaid access tokens (secure)       | Service role only                                    |
| <code>bank_accounts</code>          | Synced bank accounts               | <code>is_owner()</code> only                         |
| <code>bank_transactions</code>      | Synced transactions                | <code>is_owner()</code> only                         |
| <code>cashflow_sync_logs</code>     | Sync operation audit log           | <code>is_owner()</code> only                         |
| <code>role_permissions</code>       | Live permissions matrix            | <code>is_owner()</code>                              |

## 5.3 RLS Helper Functions

```
create function public.is_super_user() returns boolean as $$
    select exists (
        select 1 from public.profiles
        where id = auth.uid() and role = 'super_user'
    )
$$ language sql security definer stable;

create function public.is_owner() returns boolean as $$
    select exists (
        select 1 from public.profiles
        where id = auth.uid() and role in ('owner', 'super_user')
    )
$$ language sql security definer stable;

create function public.is_crm_admin() returns boolean as $$
    select exists (
        select 1 from public.profiles
        where id = auth.uid() and role in ('super_user','owner','admin')
    )
$$ language sql security definer stable;

create function public.is_crm_editor() returns boolean as $$
    select exists (
        select 1 from public.profiles
        where id = auth.uid() and role in ('super_user','owner','admin','member')
    )
$$ language sql security definer stable;

create function public.is_crm_reader() returns boolean as $$
    select exists (
        select 1 from public.profiles
        where id = auth.uid() and role in ('super_user','owner','admin','member','viewer')
    )
$$ language sql security definer stable;

create function public.is_bd() returns boolean as $$
    select exists (
        select 1 from public.profiles where id = auth.uid() and role = 'bd'
    )
$$ language sql security definer stable;
```

## 5.4 Key Schema Notes

- **BD data scoping:** BD users see only rows where `owner_id = auth.uid()`. This is enforced via a separate `SELECT` policy on each CRM table.
- **Partner data scoping:** Partners have no matching `SELECT` policy on CRM tables — PostgreSQL's deny-by-default RLS returns zero rows.

- `bank_connection_tokens` : Contains Plaid `access_token` values. No frontend SELECT policy exists on this table. Only the service role key (used inside Edge Functions) can read it.
- `role_permissions` : Stores the live permissions matrix as rows. Each row = `(permission_key, role, enabled)` . Only owner/super\_user can read or write.

## 6. Role & Permission System

### 6.1 Role Hierarchy

| Role                    | Tier | Description                                              |
|-------------------------|------|----------------------------------------------------------|
| <code>super_user</code> | 1    | Platform authority — unrestricted, cannot be limited     |
| <code>owner</code>      | 2    | Business intelligence — financial data, AI, AVE, all CRM |
| <code>admin</code>      | 3    | Operations control — CRM + team, no financial modules    |
| <code>member</code>     | 4    | Standard CRM — read and write all CRM data               |
| <code>viewer</code>     | 5    | Read-only across all CRM modules                         |
| <code>bd</code>         | 6    | Business Development — assigned-data only pipeline       |
| <code>partner</code>    | 7    | External — only explicitly shared data                   |

**Key rule:** `super_user` always returns `true` from `hasPermission()` . The matrix only governs the 6 editable roles (owner through partner).

### 6.2 TypeScript Types

```
export type TeamRole =
  "super_user" | "owner" | "admin" | "member" | "viewer" | "bd" | "partner"

export type EditableRole =
  "owner" | "admin" | "member" | "viewer" | "bd" | "partner"

export const EDITABLE_ROLES: EditableRole[] =
  ["owner", "admin", "member", "viewer", "bd", "partner"]
```

## 6.3 Permission Categories & Keys (35 total)

### CRM (5 keys)

|            |            |            |
|------------|------------|------------|
| crm.view   | crm.edit   | crm.delete |
| crm.export | crm.import |            |

### CASHFLOW (3 keys)

|               |                       |                           |
|---------------|-----------------------|---------------------------|
| cashflow.view | cashflow.transactions | cashflow.bank_connections |
|---------------|-----------------------|---------------------------|

### AI & INTELLIGENCE (3 keys)

|             |                |        |
|-------------|----------------|--------|
| ai.insights | ai.forecasting | ai.ave |
|-------------|----------------|--------|

### REPORTS (2 keys)

|              |                |
|--------------|----------------|
| reports.view | reports.export |
|--------------|----------------|

### NOTIFICATIONS (3 keys)

|                         |                           |                                  |
|-------------------------|---------------------------|----------------------------------|
| notifications.financial | notifications.operational | notifications.configure_personal |
|-------------------------|---------------------------|----------------------------------|

### TEAM MANAGEMENT (3 keys)

|             |             |             |
|-------------|-------------|-------------|
| team.invite | team.manage | team.remove |
|-------------|-------------|-------------|

### SETTINGS (2 keys)

|                  |                 |
|------------------|-----------------|
| settings.profile | settings.system |
|------------------|-----------------|

### SECURITY (3 keys)

|              |                   |                |
|--------------|-------------------|----------------|
| security.mfa | security.sessions | security.audit |
|--------------|-------------------|----------------|

### INTEGRATIONS (3 keys)

|                   |                        |                   |
|-------------------|------------------------|-------------------|
| integrations.view | integrations.configure | integrations.sync |
|-------------------|------------------------|-------------------|

### DEVOPS (4 keys)

|               |                 |             |                   |
|---------------|-----------------|-------------|-------------------|
| devops.deploy | devops.env_vars | devops.cicd | devops.monitoring |
|---------------|-----------------|-------------|-------------------|

## 6.4 Default Matrix (key access levels)

| Permission                       | Owner | Admin | Member | Viewer | BD | Partner |
|----------------------------------|-------|-------|--------|--------|----|---------|
| crm.view                         | ✓     | ✓     | ✓      | ✓      | ✓  | —       |
| crm.edit                         | ✓     | ✓     | ✓      | —      | ✓  | —       |
| crm.delete                       | ✓     | ✓     | —      | —      | —  | —       |
| crm.export                       | ✓     | ✓     | —      | —      | —  | —       |
| crm.import                       | ✓     | ✓     | —      | —      | —  | —       |
| cashflow.view                    | ✓     | —     | —      | —      | —  | —       |
| cashflow.transactions            | ✓     | —     | —      | —      | —  | —       |
| cashflow.bank_connections        | ✓     | —     | —      | —      | —  | —       |
| ai.insights                      | ✓     | —     | —      | —      | —  | —       |
| ai.forecasting                   | ✓     | —     | —      | —      | —  | —       |
| ai.ave                           | ✓     | —     | —      | —      | —  | —       |
| reports.view                     | ✓     | ✓     | ✓      | ✓      | —  | —       |
| reports.export                   | ✓     | ✓     | —      | —      | —  | —       |
| notifications.financial          | ✓     | —     | —      | —      | —  | —       |
| notifications.operational        | ✓     | ✓     | ✓      | ✓      | ✓  | ✓       |
| notifications.configure_personal | ✓     | ✓     | ✓      | ✓      | ✓  | ✓       |
| team.invite                      | ✓     | ✓     | —      | —      | —  | —       |
| team.manage                      | ✓     | ✓     | —      | —      | —  | —       |
| team.remove                      | ✓     | —     | —      | —      | —  | —       |
| settings.profile                 | ✓     | ✓     | ✓      | ✓      | ✓  | ✓       |
| settings.system                  | ✓     | —     | —      | —      | —  | —       |
| security.mfa                     | ✓     | —     | —      | —      | —  | —       |
| security.sessions                | —     | —     | —      | —      | —  | —       |
| security.audit                   | —     | —     | —      | —      | —  | —       |
| integrations.view                | ✓     | —     | —      | —      | —  | —       |



| Permission             | Owner | Admin | Member | Viewer | BD | Partner |
|------------------------|-------|-------|--------|--------|----|---------|
| integrations.configure | —     | —     | —      | —      | —  | —       |
| integrations.sync      | ✓     | —     | —      | —      | —  | —       |
| devops.deploy          | —     | —     | —      | —      | —  | —       |
| devops.env_vars        | —     | —     | —      | —      | —  | —       |
| devops.cicd            | —     | —     | —      | —      | —  | —       |
| devops.monitoring      | —     | —     | —      | —      | —  | —       |

*Blank = false. Super User always returns true regardless of matrix.*

## 6.5 Permission Persistence

The permissions matrix is stored in two ways:

- **Preview mode:** `localStorage` under key `avanew-crm.permissions`
- **Production:** `role_permissions` Supabase table, loaded on `usePermissions` mount via `fetchPermissionsFromDB()`, persisted on toggle via `persistPermissionsToDB()`

## 6.6 hasPermission Logic

```
export function hasPermission(
  matrix: PermissionsMatrix,
  role: TeamRole,
  key: PermissionKey
): boolean {
  if (role === "super_user") return true
  return matrix[key]?.[role as EditableRole] ?? false
}
```

## 7. Module Specifications

### 7.1 Routing Table

|                                 |                                           |                                         |
|---------------------------------|-------------------------------------------|-----------------------------------------|
| /                               | → redirect to /dashboard                  |                                         |
| /login                          | → Login page (public)                     |                                         |
| /signup                         | → Signup page (public)                    |                                         |
| /dashboard                      | → Dashboard                               |                                         |
| /leads                          | → Lead list                               |                                         |
| /leads/new                      | → Create lead                             |                                         |
| /leads/:id/edit                 | → Edit lead                               |                                         |
| /contacts                       | → Contact list                            |                                         |
| /contacts/new                   | → Create contact                          |                                         |
| /contacts/:id/edit              | → Edit contact                            |                                         |
| /companies → /accounts          | → redirect                                |                                         |
| /accounts                       | → Account (company) list                  |                                         |
| /accounts/new                   | → Create account                          |                                         |
| /accounts/:id/edit              | → Edit account                            |                                         |
| /deals                          | → Deal kanban + list                      |                                         |
| /deals/new                      | → Create deal                             |                                         |
| /deals/:id/edit                 | → Edit deal                               |                                         |
| /tasks                          | → Task list                               |                                         |
| /tasks/new                      | → Create task                             |                                         |
| /tasks/:id/edit                 | → Edit task                               |                                         |
| /activities                     | → Activity log                            |                                         |
| /reports                        | → Reports dashboard                       |                                         |
| /cashflow                       | → Cashflow dashboard                      | [permission: cashflow.view]             |
| /cashflow/transactions          | → Transaction ledger                      | [permission: cashflow.transactions]     |
| /cashflow/transactions/new      | → Create transaction                      | [permission: cashflow.transactions]     |
| /cashflow/transactions/:id/edit | → Edit transaction                        | [permission: cashflow.transactions]     |
| /cashflow/bank-connections      | → Bank connections                        | [permission: cashflow.bank_connections] |
| /partners                       | → Partner list                            |                                         |
| /partners/new                   | → Create partner                          |                                         |
| /partners/:id/edit              | → Edit partner                            |                                         |
| /vendors                        | → Vendor list                             |                                         |
| /vendors/new                    | → Create vendor                           |                                         |
| /vendors/:id/edit               | → Edit vendor                             |                                         |
| /permissions                    | → redirect to /settings/roles             |                                         |
| /settings                       | → redirect to /settings/profile           |                                         |
| /settings/profile               | → User Management                         |                                         |
| /settings/roles                 | → Roles & Permissions (PermissionsMatrix) |                                         |
| /settings/company               | → Company Profile                         | [allow: owner, super_user, admin]       |
| /settings/team                  | → Team / Organization                     | [allow: owner, super_user, admin]       |
| /settings/pipeline              | → Pipeline Settings                       | [allow: owner, super_user, admin]       |
| /settings/financial             | → Financial Settings                      | [permission: cashflow.bank_connections] |
| /settings/partners-vendors      | → Partner & Vendor Settings               | [allow: owner, super_user, admin]       |
| /settings/integrations          | → Integrations                            | [allow: super_user]                     |
| /settings/notifications         | → Notifications                           | [all roles]                             |
| /settings/security              | → Security                                | [allow: owner, super_user]              |
| /settings/audit-logs            | → Audit Logs                              | [allow: owner, super_user]              |

|                    |                      |                            |
|--------------------|----------------------|----------------------------|
| /settings/data     | → Data Management    | [allow: super_user]        |
| /settings/branding | → Branding / UI      | [allow: owner, super_user] |
| /settings/system   | → System Preferences | [allow: super_user]        |

## 7.2 AppSidebar Navigation Structure

```

Avanew Command Center [logo]
-----
Dashboard
----- MODULES -----
▶ CRM (collapsible)
  Leads / Contacts / Accounts / Deals / Activities / Reports
Tasks
▶ Cashflow (collapsible) [visible if can("cashflow.view")]
  Dashboard / Transactions / Bank Connections
----- RELATIONSHIPS -----
Partners
Vendors
----- SYSTEM -----
Settings
-----
v0.1 · ACC

```

## 7.3 CRM Module

All CRM tables share the same access pattern:

- **Readers** (viewer+): read all rows
- **Editors** (member+): create and update
- **Admins** (admin+): delete
- **BD**: read/write only rows where `owner_id = auth.uid()`
- **Partner**: no access (zero rows returned by RLS)

CRM pages follow a consistent pattern:

1. List page with search, filter, and action buttons
2. Form page for create/edit (same component, parameterized by route `:id`)
3. Navigation uses `PageHeader` with title + optional action buttons

## 7.4 Cashflow Module

The Cashflow module is restricted to Owner and Super User via `is_owner()` RLS.

### Components:

- `Cashflow.tsx` — Dashboard with financial health KPIs, summaries

- `Transactions.tsx` — Full transaction ledger with category filter
- `TransactionForm.tsx` — Manual transaction entry
- `BankConnections.tsx` — Manage Mercury/Plaid connections + sync

**Transaction classification** ( `src/lib/transaction-classifier.ts` ): Uses regex pattern matching on description + merchant name + raw category to assign one of: `Transfer` | `Payroll` | `Tax Payment` | `Infrastructure` | `Software` | `Loan / Financing` | `Refund` | `Owner Contribution` | `Contractor` | `Vendor Payment` | `Revenue` | `Expense`

## 7.5 Bank Connections

Two providers are supported:

| Provider | Connection Method     | Edge Function                                                                                       |
|----------|-----------------------|-----------------------------------------------------------------------------------------------------|
| Mercury  | API Key (server-side) | <code>mercury-sync</code>                                                                           |
| Plaid    | OAuth Link Token flow | <code>plaid-link-token</code> → <code>plaid-sync</code> (exchange) → <code>plaid-sync</code> (sync) |

Connection records in `bank_connections` store provider, institution name, status, and `last_sync_at`. The Plaid `access_token` is stored exclusively in `bank_connection_tokens` — inaccessible to the frontend by design.

## 8. Settings Module

### 8.1 Architecture

The Settings module uses React Router nested routes. `SettingsLayout` renders a two-column layout: a persistent left sidebar (desktop) or Sheet drawer (mobile), plus an `<Outlet />` for the active child page.

```

/settings → <SettingsLayout>
├─ /settings/profile → <SettingsProfile />
├─ /settings/roles → <SettingsRoles /> (PermissionsMatrix)
├─ /settings/company → <SettingsCompany />
├─ /settings/team → <SettingsTeam />
├─ /settings/pipeline → <SettingsPipeline />
├─ /settings/financial → <SettingsFinancial />
├─ /settings/partners-vendors → <SettingsPartnersVendors />
├─ /settings/integrations → <SettingsIntegrations />
├─ /settings/notifications → <SettingsNotifications />
├─ /settings/security → <SettingsSecurity />
├─ /settings/audit-logs → <SettingsAuditLogs />
├─ /settings/data → <SettingsData />
├─ /settings/branding → <SettingsBranding />
└─ /settings/system → <SettingsSystem />

```

## 8.2 Settings Sidebar Nav Groups

```

ACCOUNT
  User Management          (all roles)
  Company Profile          (owner, super_user, admin)
  Team / Organization      (owner, super_user, admin)

CONFIGURATION
  Pipeline / Deal Settings (owner, super_user, admin)
  Financial Settings       (owner, super_user)
  Partner & Vendor Settings (owner, super_user, admin)
  Roles & Permissions      (owner, super_user)

PREFERENCES
  Notifications            (all roles)
  Branding / UI Settings   (owner, super_user)

SYSTEM
  Integrations             (super_user only)
  Security                 (owner, super_user)
  Audit Logs               (owner, super_user)
  Data Management          (super_user only)
  System Preferences       (super_user only)

```

## 8.3 Responsive Behavior

| Breakpoint      | Behavior                                                                             |
|-----------------|--------------------------------------------------------------------------------------|
| < 1g (< 1024px) | Mobile: Sheet drawer triggered by a button showing current section name + breadcrumb |
| ≥ 1g (1024px+)  | Desktop: Persistent sticky sidebar ( <code>sticky top-0 self-start w-52</code> )     |

## 8.4 Content Status

| Page                      | Status                 | Migrated From                                |
|---------------------------|------------------------|----------------------------------------------|
| User Management           | Implemented            | Old Settings.tsx profile form + role cards   |
| Roles & Permissions       | Implemented            | Full PermissionsMatrix component             |
| Company Profile           | Placeholder            | —                                            |
| Team / Organization       | Implemented            | TeamSection component                        |
| Pipeline / Deal Settings  | Implemented            | StageManager button                          |
| Financial Settings        | Implemented            | Bank Integrations card from old Settings.tsx |
| Partner & Vendor Settings | Placeholder            | —                                            |
| Integrations              | Structured placeholder | Shows 5 integration slots                    |
| Notifications             | Role-aware             | Shows available notification types per role  |
| Security                  | Structured placeholder | MFA + Login History cards                    |
| Audit Logs                | Placeholder            | —                                            |
| Data Management           | Structured placeholder | Import/Export/Cleanup cards                  |
| Branding / UI Settings    | Structured placeholder | Theme/Dark Mode/Typography cards             |
| System Preferences        | Structured placeholder | Locale/Session/Feature Flags                 |

## 9. Edge Functions & Backend Services

### 9.1 Shared Auth Middleware

**File:** `supabase/functions/_shared/auth.ts`

All Edge Functions use the centralized `requireAuth()` helper:

```

export async function requireAuth(
  req: Request,
  allowedRoles?: string[]
): Promise<AuthResult>

// Returns:
// { ok: true, profile: { id, role, email }, supabase: SupabaseClient }
// { ok: false, response: Response } ← 401 or 403 JSON response

// Usage pattern in every Edge Function:
const auth = await requireAuth(req, ["owner", "super_user"])
if (!auth.ok) return auth.response
const { profile, supabase } = auth

```

The helper:

1. Validates the `Authorization: Bearer <token>` header
2. Calls `supabase.auth.getUser(token)`
3. Fetches `profiles` row for `id, role, email`
4. Checks `allowedRoles` if provided
5. Returns typed `AuthResult`

## 9.2 mercury-sync

**Endpoint:** `POST /functions/v1/mercury-sync`

**Auth:** Owner or Super User

**Body:** `{ connection_id: string }`

Flow:

1. Validates connection is a Mercury provider record
2. Fetches all accounts via `GET /api/v1/accounts`
3. Upserts to `bank_accounts` table
4. Fetches transactions since `last_sync_at` (or 90 days on first run)
5. Upserts to `bank_transactions` with inline classification
6. Updates `bank_connections.status` and `last_sync_at`
7. Writes audit row to `cashflow_sync_logs`

## 9.3 plaid-link-token

**Endpoint:** `POST /functions/v1/plaid-link-token`

**Auth:** Owner or Super User

**Body:** None

Flow:

1. Calls Plaid `/link/token/create` with `products: ["transactions"]`
2. Returns `{ link_token: string }` to frontend
3. Frontend uses `link_token` with Plaid Link SDK to obtain `public_token`

## 9.4 plaid-sync

**Endpoint:** `POST /functions/v1/plaid-sync`

**Auth:** Owner or Super User

### Action: exchange

Body: `{ action: "exchange", public_token: string, institution_name: string }`

1. Calls Plaid `/item/public_token/exchange`
2. Inserts `bank_connections` row (no `access_token` column)
3. Inserts `bank_connection_tokens` row with `access_token` (service role only)
4. Returns `{ connection_id: string }`

### Action: sync

Body: `{ action: "sync", connection_id: string }`

1. Reads `access_token` from `bank_connection_tokens` (service role, never frontend)
2. Fetches accounts via Plaid `/accounts/get`
3. Fetches transactions via Plaid `/transactions/get` (paginated, 500/page)
4. Upserts all data with classification
5. Updates connection status and logs

## 9.5 ave-session

**Endpoint:** `POST /functions/v1/ave-session`

**Auth:** Owner or Super User

**Tavus config:** `audio_only: true` , `participant_left_timeout: 300` , `enable_recording: false` ,  
`max_call_duration: 3600`

### Action: start

Body: `{ action: "start", context?: string }`

1. Calls Tavus `POST /v2/conversations` with `replica_id` and optional `persona_id`
2. Sets `custom_greeting` to context if provided
3. Returns `{ conversation_id, conversation_url }`

### Action: end

Body: `{ action: "end", conversation_id: string }`

1. Calls Tavus `POST /v2/conversations/{id}/end`
2. Returns `{ ok: true }`



---

## 10. Key Components & Hooks

---

### 10.1 useRole

```
// src/hooks/useRole.ts
export function useRole(): {
  role: TeamRole | null
  isOwner: boolean // role === "owner" || role === "super_user"
  isSuperUser: boolean // role === "super_user"
  loading: boolean
}
```

In preview mode: returns `super_user` synchronously. In production: fetches from `profiles` table via Supabase auth.

### 10.2 usePermissions

```
// src/hooks/usePermissions.ts
export function usePermissions(): {
  can: (key: PermissionKey) => boolean
  matrix: PermissionsMatrix
  toggle: (key: PermissionKey, targetRole: string, value: boolean) => void
  reset: () => void
}
```

On mount (production): fetches matrix from `role_permissions` table and hydrates state. On toggle: updates state + persists to DB.

### 10.3 useAVE

```
// src/hooks/useAVE.ts
type AVEStatus = "idle" | "starting" | "active" | "ending" | "error"

export function useAVE(): {
  status: AVEStatus
  conversationUrl: string | null
  errorMsg: string | null
  elapsed: number // seconds since session started
  start: (context: string) => Promise<void>
  end: () => Promise<void>
}
```

State machine transitions: `idle → starting → active → ending → idle` . Error state is reachable from `starting` . Elapsed timer runs on an interval while `active` .

## 10.4 RoleGate

```
// src/components/RoleGate.tsx
<RoleGate allow={"owner", "super_user"}> { /* role-list check */}
<RoleGate permission="cashflow.view"> { /* matrix check */}
```

In preview mode: always renders children. Otherwise: checks role/permission, redirects to `/dashboard` if unauthorized.

## 10.5 AVE Component

The AVE floating button renders in `AppLayout.tsx` (always present). It gates on `can("ai.ave")` . When active:

- Shows a panel with green live indicator, elapsed timer, context module name, animated waveform
- Embeds a hidden `<iframe src={conversationUrl} allow="microphone; autoplay">` for Tavus Daily.co WebRTC audio
- Shows an End Session button

## 10.6 PermissionsMatrix Page

A full-page table showing 35 permissions × 7 roles. Super User column shows lock icons (always true). Editable role columns show toggle switches. Owner/Super User can toggle any non-super\_user cell. A "Reset to defaults" button (with confirmation dialog) restores `DEFAULT_MATRIX` .

Summary strip above the table shows enabled-count and percentage per role. Category rows act as separators. Permission keys are shown in monospace below each permission label.

---

# 11. AVE — AI Voice Experience

## 11.1 Overview

AVE (AI Voice Experience) is a floating voice assistant that lets the Owner and Super User have a natural language conversation with an AI that has full context of the current application module and the business's CRM + financial data.

## 11.2 Architecture

```
User taps AVE button
  → useAVE.start(currentModule)
  → POST /functions/v1/ave-session { action: "start", context: "Deals" }
  → Tavus creates conversation (audio_only: true)
  → Returns { conversation_id, conversation_url }
  → <iframe src={conversation_url} allow="microphone; autoplay">
    (Tavus Daily.co room handles WebRTC audio)
  → AVE panel shows: live indicator, timer, context, waveform

User taps End Session
  → useAVE.end()
  → POST /functions/v1/ave-session { action: "end", conversation_id }
  → Tavus closes the conversation
  → State returns to idle
```

## 11.3 Context Injection

The `MODULE_LABELS` map in `AVE.tsx` maps pathname prefixes to human-readable module names:

```
/dashboard → "Dashboard"
/leads     → "Leads"
/contacts  → "Contacts"
/accounts  → "Accounts"
/deals     → "Deals"
/tasks     → "Tasks"
/activities → "Activities"
/reports   → "Reports"
/cashflow  → "Cashflow"
/partners  → "Partners"
/vendors   → "Vendors"
/settings  → "Settings"
/permissions → "Permissions"
```

The `custom_greeting` field in the Tavus API payload receives: `"The user is currently viewing: {module}."`

## 11.4 Required Supabase Secrets

```
TAVUS_API_KEY      - from Tavus dashboard
TAVUS_REPLICA_ID   - replica ID for the AVE persona
TAVUS_PERSONA_ID   - (optional) persona ID for custom context injection
```

## 12. Environment Configuration

### 12.1 Frontend Environment Variables

`.env` (committed — public keys only):

```
VITE_SUPABASE_URL=https://your-project.supabase.co
VITE_SUPABASE_ANON_KEY=your-anon-key
```

`.env.local` (not committed — dev overrides):

```
VITE_PREVIEW_MODE=true
```

### 12.2 Supabase Edge Function Secrets

Set in Supabase Dashboard → Edge Functions → Secrets:

| Secret                                 | Used By      | Description                                 |
|----------------------------------------|--------------|---------------------------------------------|
| <code>MERCURY_API_KEY</code>           | mercury-sync | Read-only Mercury API token                 |
| <code>PLAID_CLIENT_ID</code>           | plaid-*      | Plaid application client ID                 |
| <code>PLAID_SECRET</code>              | plaid-*      | Plaid sandbox/development/production secret |
| <code>PLAID_ENV</code>                 | plaid-*      | "sandbox"   "development"   "production"    |
| <code>TAVUS_API_KEY</code>             | ave-session  | Tavus API key                               |
| <code>TAVUS_REPLICA_ID</code>          | ave-session  | Tavus replica ID                            |
| <code>TAVUS_PERSONA_ID</code>          | ave-session  | (optional) Tavus persona ID                 |
| <code>SUPABASE_URL</code>              | all          | Auto-injected by Supabase runtime           |
| <code>SUPABASE_SERVICE_ROLE_KEY</code> | all          | Auto-injected by Supabase runtime           |

### 12.3 Supabase Project Setup Checklist

1. Create Supabase project
2. Run `supabase/schema.sql` in the SQL editor
3. Enable Row Level Security on all tables (schema.sql handles this)
4. Configure Auth → Email provider
5. Set Edge Function secrets (section 12.2)
6. Deploy Edge Functions:

```
supabase functions deploy mercury-sync
supabase functions deploy plaid-link-token
supabase functions deploy plaid-sync
supabase functions deploy ave-session
```

7. Create the first `super_user` profile manually in the profiles table after signup

---

## 13. Deployment Guide

---

### 13.1 Local Development

```
# Install dependencies
npm install

# Start with preview mode (no Supabase needed)
VITE_PREVIEW_MODE=true npm run dev

# Start with live Supabase
npm run dev
```

### 13.2 Build

```
npm run build
# Output: dist/
```

### 13.3 GitHub Repository Setup

```
git init
git add .
git commit -m "feat: Avaneew Command Center v1"
git remote add origin https://github.com/your-org/avaneew-crm.git
git push -u origin main
```

Required `.gitignore` entries:

```
.env.local
node_modules/
dist/
```

## 13.4 AWS EC2 Deployment

```
# On EC2 instance (Amazon Linux 2 / Ubuntu)
# Install Node.js 20+
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs

# Clone repo
git clone https://github.com/your-org/avanew-crm.git
cd avanew-crm

# Set environment variables
echo "VITE_SUPABASE_URL=https://your-project.supabase.co" > .env
echo "VITE_SUPABASE_ANON_KEY=your-anon-key" >> .env

# Build
npm install
npm run build

# Serve with nginx or PM2 + serve
npm install -g serve
pm2 start "serve -s dist -l 3000" --name avanew-crm
```

### Nginx reverse proxy config:

```
server {
    listen 80;
    server_name your-domain.com;
    root /home/ubuntu/avanew-crm/dist;
    index index.html;
    location / {
        try_files $uri /index.html;
    }
}
```

---

## 14. Master Reproduction Prompt

The following prompt, when provided to Claude Code (Sonnet 4.5+) in a fresh session with the Avaneew CRM project directory open, will reproduce this application. Split into phases for manageability.

---

## MASTER PROMPT — AVANEW COMMAND CENTER V1

You are building Avaneu Command Center, a full-stack internal CRM and business intelligence platform. Use the following specification exactly.

---

### TECH STACK

---

Frontend: React 19, TypeScript (strict), Vite, Tailwind CSS v4, shadcn/ui  
Backend: Supabase (PostgreSQL + Auth + Edge Functions + RLS)  
Icons: Lucide React  
Toast: Sonner  
Routing: React Router DOM v6  
DnD: @dnd-kit (for kanban)  
Language: TypeScript throughout. No JavaScript files.

---

### PREVIEW MODE

---

VITE\_PREVIEW\_MODE=true bypasses all auth and Supabase calls, uses mock data.  
Check: `const PREVIEW_MODE = import.meta.env.VITE_PREVIEW_MODE === "true"`  
In preview: `role = "super_user"`, mock data from `src/lib/data.ts`,  
`localStorage` for permissions persistence.

---

### ROLE SYSTEM

---

```
type TeamRole = "super_user"|"owner"|"admin"|"member"|"viewer"|"bd"|"partner"
type EditableRole = Exclude<TeamRole, "super_user">
```

7-tier hierarchy:

- `super_user`: unrestricted platform authority, always returns true from `hasPermission`
- `owner`: full business intelligence (financial, AI, AVE, all CRM)
- `admin`: operations control (CRM + team, no financial)
- `member`: standard CRM read/write
- `viewer`: read-only CRM
- `bd`: assigned-data-only pipeline (`owner_id = auth.uid()` filter)
- `partner`: external, only explicitly shared data

```
isOwner = role === "owner" || role === "super_user"
```

---

### PERMISSIONS MATRIX — 35 keys × 6 editable roles

---

Categories and keys:

CRM: `crm.view`, `crm.edit`, `crm.delete`, `crm.export`, `crm.import`  
CASHFLOW: `cashflow.view`, `cashflow.transactions`, `cashflow.bank_connections`  
AI: `ai.insights`, `ai.forecasting`, `ai.ave`  
REPORTS: `reports.view`, `reports.export`  
NOTIFICATIONS: `notifications.financial`, `notifications.operational`,  
`notifications.configure_personal`

```
TEAM: team.invite, team.manage, team.remove
SETTINGS: settings.profile, settings.system
SECURITY: security.mfa, security.sessions, security.audit
INTEGRATIONS: integrations.view, integrations.configure, integrations.sync
DEVOPS: devops.deploy, devops.env_vars, devops.cicd, devops.monitoring
```

Default access (true):

```
owner: everything EXCEPT security.sessions, security.audit,
      integrations.configure, devops.*
admin: crm.*, reports.*, notifications.operational,
      notifications.configure_personal, team.invite, team.manage,
      settings.profile, settings.system
member: crm.view, crm.edit, reports.view, notifications.operational,
        notifications.configure_personal, settings.profile
viewer: crm.view, reports.view, notifications.operational,
        notifications.configure_personal, settings.profile
bd: crm.view, crm.edit, notifications.operational,
    notifications.configure_personal, settings.profile
partner: notifications.operational, notifications.configure_personal,
        settings.profile
```

Persistence: localStorage in preview, role\_permissions Supabase table in production.  
 Load on mount via fetchPermissionsFromDB(), save on toggle via persistPermissionsToDB().  
 Matrix stored as rows: (permission\_key, role, enabled).

---

#### DATABASE SCHEMA — KEY TABLES

---

```
profiles: id (uuid, refs auth.users), email, full_name, role (team_role),
          avatar_url, created_at
leads: id, first_name, last_name, email, phone, company, status, source,
       notes, owner_id, assigned_to, created_at, updated_at
contacts: id, first_name, last_name, email, phone, company_id, title,
          notes, owner_id, created_at, updated_at
companies: id, name, industry, website, phone, address, notes,
           owner_id, created_at, updated_at
deals: id, title, value, stage_id, contact_id, company_id, owner_id,
       probability, close_date, notes, created_at, updated_at
activities: id, type (call|email|meeting|note), title, notes,
            contact_id, deal_id, owner_id, occurred_at, created_at
tasks: id, title, description, status (todo|in_progress|done),
       priority (low|medium|high), due_date, owner_id, assigned_to,
       contact_id, deal_id, created_at, updated_at
pipeline_stages: id, name, order_index, is_won, is_lost, color, created_at
partners: id, name, type, contact_name, email, phone, status, notes, created_at
vendors: id, name, category, contact_name, email, phone, status, notes, created_at
cashflow_transactions: id, date, description, amount, category, type
                      (income|expense), notes, created_at
bank_connections: id, provider (mercury|plaid), institution_name, institution_id,
                  status (pending|active|error), last_sync_at, error_message,
                  created_at, updated_at
bank_connection_tokens: id, connection_id, access_token, created_at
```



```

bank_accounts: id, bank_connection_id, external_account_id, name, type,
               subtype, balance_current, balance_available, currency,
               institution_name, is_active, last_updated
bank_transactions: id, bank_connection_id, bank_account_id, provider,
                  external_transaction_id, date, description, amount,
                  currency, category, pending, merchant_name, raw_category,
                  is_excluded, created_at
cashflow_sync_logs: id, bank_connection_id, provider, status,
                   transactions_imported, transactions_skipped,
                   error_message, started_at, completed_at
role_permissions: permission_key (text), role (text), enabled (boolean),
                 updated_at - PK: (permission_key, role)

```

RLS helpers (security definer functions):

```

is_super_user() - role = 'super_user'
is_owner()      - role in ('owner', 'super_user')
is_crm_admin()  - role in ('super_user', 'owner', 'admin')
is_crm_editor() - role in ('super_user', 'owner', 'admin', 'member')
is_crm_reader() - role in ('super_user', 'owner', 'admin', 'member', 'viewer')
is_bd()         - role = 'bd'

```

```

CRM tables: SELECT for is_crm_reader() OR (is_bd() AND owner_id=auth.uid())
            INSERT/UPDATE for is_crm_editor() OR (is_bd() AND own row)
            DELETE for is_crm_admin()

```

Financial tables (cashflow\_\*, bank\_\*): ALL for is\_owner()

role\_permissions: SELECT/ALL for is\_owner()

bank\_connection\_tokens: service role only (no RLS policies for frontend)

---

## APPLICATION LAYOUT

---

AppLayout: flex row - AppSidebar (hidden md, w-64) + flex col (TopBar + main)

main: flex-1 overflow-y-auto p-4 md:p-6

AVE floating button renders inside AppLayout, outside main

AppSidebar sections:

```

Logo: "AC" square + "Avanew Command / Center"
Nav: Dashboard | MODULES header | CRM (collapsible) | Tasks |
    Cashflow (collapsible, if can("cashflow.view")) |
    RELATIONSHIPS header | Partners | Vendors |
    SYSTEM header | Settings
Footer: "v0.1 · ACC"

```

CRM collapsible children: Leads, Contacts, Accounts, Deals, Activities, Reports

Cashflow collapsible children: Dashboard, Transactions, Bank Connections

TopBar: hamburger (mobile), theme toggle, user avatar button with dropdown  
(profile link, sign out)

---

## SETTINGS MODULE - NESTED ROUTES

---

/settings → SettingsLayout (desktop sidebar + Outlet)  
/settings/\* → 14 child routes (see routing table in spec)

#### SettingsLayout:

Desktop (lg+): sticky left sidebar w-52 with grouped nav  
Mobile (<lg): Sheet drawer triggered by button showing current section

Nav groups: Account | Configuration | Preferences | System

Filter items by role using canSee(item, role) — super\_user sees all.

---

#### EDGE FUNCTIONS — PATTERN

---

All functions use \_shared/auth.ts:

```
const auth = await requireAuth(req, ["owner", "super_user"])  
if (!auth.ok) return auth.response  
const { profile, supabase } = auth
```

profile has: { id: string, role: string, email: string | null }

\_shared/cors.ts exports corsHeaders object.

All functions handle OPTIONS preflight:

```
if (req.method === "OPTIONS") return new Response("ok", { headers: corsHeaders })
```

Functions: mercury-sync, plaid-link-token, plaid-sync, ave-session

(see section 9 of specification for full details)

---

#### AVE COMPONENT

---

Gate: can("ai.ave") — NOT isOwner

State machine: "idle"|"starting"|"active"|"ending"|"error"

Floating bottom-right panel:

- FAB button always visible
- Active panel: green pulse dot, elapsed MM:SS, context module, animated waveform
- Hidden iframe: <iframe src={conversationUrl} allow="microphone; autoplay">
- Error panel with retry button

Context injection: POST { action: "start", context: currentModuleName }

Waveform animation: 10 bars, varying heights, CSS keyframes ave-bar

---

#### KEY HOOKS

---

useRole(): { role, isOwner, isSuperUser, loading }

Preview: returns super\_user immediately

Production: fetches from profiles via auth.getUser()

usePermissions(): { can(key), matrix, toggle(key, role, value), reset() }

Preview: localStorage persistence

Production: Supabase role\_permissions table

useAVE(): { status, conversationUrl, errorMsg, elapsed, start(context), end() }

Preview: 1.4s fake delay then active state

---

## COMPONENTS — KEY PATTERNS

---

### RoleGate:

<RoleGate allow={["owner","super\_user"]}> — role list check

<RoleGate permission="cashflow.view"> — matrix check

Preview mode: always renders children

Otherwise: redirects to /dashboard if unauthorized

PageHeader: title + optional description + optional actions (right-aligned)

Controlled widths: max-w-3xl for forms, max-w-4xl for wider content

No full-width layouts on forms.

### PermissionsMatrix:

Table: 35 rows × 7 columns (Permission | Super User | 6 editable roles)

Super User column: lock icons (always true, not togglable)

Editable columns: toggle switches

Category headers: colspan 8, uppercase label

Summary strip above table: enabled count + % per role

Reset button with AlertDialog confirmation

---

## TRANSACTION CLASSIFIER

---

Input: { description, merchant\_name?, raw\_category?, amount }

Output: one of Transfer | Payroll | Tax Payment | Infrastructure | Software |  
Loan/Financing | Refund | Owner Contribution | Contractor |  
Vendor Payment | Revenue | Expense

### Regex patterns (applied in order):

transfer|wire|zelle|internal → Transfer

payroll|gusto|rippling|adp|paychex|salary|wages → Payroll

irs|state tax|federal tax|estimated tax|quarterly tax → Tax Payment

aws|google cloud|digitalocean|cloudflare|datadog → Infrastructure

github|figma|slack|notion|linear|vercel|stripe|zoom|atlassian → Software

loan|credit facility|line of credit|financing → Loan/Financing

refund|reversal|chargeback → Refund

amount > 0 + owner contribution|capital contribution → Owner Contribution

amount < 0 + contractor|freelance|consulting fee|1099 → Contractor

amount < 0 + vendor|supplier|invoice → Vendor Payment

amount > 0 → Revenue | amount < 0 → Expense

---

## BUILD ORDER (RECOMMENDED)

---

### Phase 1: Project scaffold

- Vite + React + TypeScript + Tailwind v4
- Install shadcn/ui components
- Set up src/ directory structure
- src/lib/supabase.ts client singleton

- src/types/db.ts with all types

#### Phase 2: Auth + Layout

- Login / Signup pages
- ProtectedRoute + AppLayout
- AppSidebar + TopBar
- useRole hook + preview mode

#### Phase 3: Core CRM

- supabase/schema.sql (all tables + RLS)
- src/lib/data.ts (all CRUD functions + mock data)
- All CRM pages: Leads, Contacts, Companies, Deals, Activities, Tasks, Reports
- PageHeader component
- StageManager dialog

#### Phase 4: Cashflow Module

- Cashflow, Transactions, TransactionForm, BankConnections pages
- src/lib/transaction-classifier.ts
- BankConnections page with Mercury + Plaid UI

#### Phase 5: Partners, Vendors, Dashboard

#### Phase 6: Role & Permissions System

- src/lib/permissions.ts (full matrix + 35 keys + helpers)
- usePermissions hook
- RoleGate component (allow + permission props)
- PermissionsMatrix page
- Wire all cashflow routes through RoleGate

#### Phase 7: Edge Functions

- supabase/functions/\_shared/auth.ts + cors.ts
- mercury-sync, plaid-link-token, plaid-sync, ave-session

#### Phase 8: AVE

- useAVE hook (state machine)
- AVE component (FAB + panel + iframe)
- Add <AVE /> to AppLayout

#### Phase 9: Settings Module

- All 15 files in src/pages/settings/
- Wire nested routes in App.tsx
- Move Settings link into main nav (SYSTEM section)

#### Phase 10: Final wiring

- role\_permissions table in schema.sql
- Production persistence in usePermissions
- /permissions redirect to /settings/roles
- Verify all four enforcement layers on every route

---

#### QUALITY REQUIREMENTS

---

- No TypeScript errors (run `tsc --noEmit` before each phase)
- No console errors in preview mode
- All forms use controlled-width containers (`max-w-3xl` or `max-w-4xl`)
- All sensitive routes protected at UI + Route + API + Data layers
- BD users see ONLY their assigned records across all CRM tables
- Partner users see ZERO CRM records (no SELECT policy = zero rows)
- `bank_connection_tokens` is NEVER readable from the frontend
- `super_user` ALWAYS bypasses the permissions matrix
- Preview mode gives full UI access without any real credentials

*End of Master Reproduction Prompt*

## Document Revision History

| Version | Date       | Author             | Changes                                     |
|---------|------------|--------------------|---------------------------------------------|
| 1.0.0   | 2026-04-28 | Avanew Engineering | Initial release — full system specification |

*Avanew Command Center · Version 1.0.0 · Confidential*